DAY 15 | HANDS-ON ACTIVITIES

Testing and Data Engineering Intro: Problem Statements

Seven problems: four on testing, three on data pipelines. Run PyTest with !python -m pytest -q in Colab or pytest -q in the VS Code terminal.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: First Tests

Level: Warm-up | **Concepts:** assert, pytest.raises

Write a module with is_valid_email(s), discount(price, pct) and grade(marks). Write at least two tests per function, including one error case.

Sample run / expected result

```
pytest -q
7 passed in 0.03s
```

Hints

- Name files test_*.py and functions test_*.

Problem 2: Parametrize the Boundaries

Level: Easy | **Concepts:** pytest.mark.parametrize

Test grade(marks) with one parametrized test covering 0, 49, 50, 69, 70, 84, 85 and 100, plus a test that 101 raises ValueError.

Sample run / expected result

```
9 passed
```

Hints

- Boundary values are where bugs hide.

Problem 3: Ticket API Test Suite

Level: Medium | **Concepts:** fixtures, TestClient

Write a client fixture and 8 tests for a Tickets API: list, create (201), invalid create (422), get, missing (404), patch status, delete (204), delete twice (404).

Sample run / expected result

```
8 passed
```

Hints

- Reset the in-memory store in a fixture so tests do not affect each other.

Problem 4: Review a Codex-Drafted Test File NEW (v4)

Level: Medium | **Concepts:** AI-assisted testing, review

The notebook contains a test file drafted by an AI assistant for sla_minutes(priority). Run it, then review every test against the written rules. For each test, mark keep, fix or delete, with a reason.

Then ask Codex (or use the notebook's online cell) to draft tests for your own function, and repeat the review.

Sample run / expected result

```
drafted: 6 tests -> 4 passed, 2 failed
review: keep 3, fix 2 (wrong expectation), delete 1 (no assert)
after review: 8 passed
```

Hints

- A failing test can be right. Check the rule before changing the code.
- Note the prompt you used and roughly how many tokens it took.

Problem 5: Messy Data ETL

Level: Medium | **Concepts:** pandas extract, transform, load

Given legacy_tickets.csv with duplicates, blank minutes, mixed date formats and extra spaces, write extract, transform and load functions that produce a clean tickets table in SQLite.

Sample run / expected result

```
Raw rows: 12 -> Clean rows: 8
Loaded 8 rows into smartdesk.db.tickets
```

Hints

- drop_duplicates, str.strip, pd.to_numeric(errors="coerce"), pd.to_datetime(format="mixed")

Problem 6: ELT Version

Level: Challenging | **Concepts:** load raw, transform with SQL

Load the raw CSV as-is into a raw_tickets table, then create a clean tickets_clean table with a SQL query inside SQLite. Compare row counts with your ETL result.

Sample run / expected result

```
raw_tickets: 12 rows
tickets_clean: 8 rows (matches ETL)
```

Hints

- CREATE TABLE tickets_clean AS SELECT DISTINCT ... WHERE minutes IS NOT NULL

Problem 7: Idempotent Load with Quality Tests

Level: Stretch | **Concepts:** incremental ingestion, data quality

Write load_incremental(new_df, conn) that inserts only tickets not already present. Prove that running it twice keeps the same row count, and add PyTest data quality tests (unique ids, valid priorities, no negative minutes).

Sample run / expected result

```
Run 1: inserted 3
Run 2: inserted 0
4 passed
```

Hints

- Read existing ids with pd.read_sql, then filter with ~df.ticket_id.isin(ids).